
CampusNest

Smart Hostel Management System

Group Project Report · Group 41 · Academic Year 2024/2025

Group Members	
254156M	Perera M. I. P
254157R	Perera O. S. G
254158V	Perera P.B.H.N.S
254159B	Perera P.S.K
254160U	Perera R.K.O.C

University of Moratuwa · Faculty of Information Technology

1. Project Title

CampusNest – Smart Hostel Management System

Console-based C Application · Group 41 · Academic Year 2024/2025

CampusNest is a console-based application developed in pure C (C99 standard) that provides a centralised digital solution for managing university hostel operations. The system addresses the operational inefficiencies of manual record keeping by offering an integrated, menu-driven platform to manage students, rooms, fees, visitors, and maintenance requests.

2. Group Members and Individual Contributions

Each of the five group members was assigned one major module of the system, ensuring equal and clearly delineated contributions across the project.

Registration No.	Name	Module	Key Responsibilities
254156M	Perera M. I. P	Student Management	Student registration, search, update, delete, and sorting operations
254157R	Perera O. S. G	Room Management	Room allocation, availability tracking, status updates, and management
254158V	Perera P.B.H.N.S	Fee Management	Fee recording, payment status updates, fee summaries, and reporting
254159B	Perera P.S.K	Visitor Management	Visitor registration, tracking, search functions, and visitor reports
254160U	Perera R.K.O.C	Maintenance Management	Maintenance request handling, status tracking, priority management

3. System Overview

3.1 Problem Context

University hostels traditionally rely on paper-based systems for managing daily operations. This approach introduces three core challenges:

Challenge	Description
Manual Record Keeping	All student and room data is tracked on paper, causing delays and increased risk of human error.
Inconsistent Data	Multiple paper registers lead to conflicting records and lost information across departments.
Slow Response Times	Visitor logs, fee tracking, and maintenance requests take hours to process and locate.

3.2 Problem Statement

University hostels lack a centralised, digital system to manage students, rooms, fees, visitors, and maintenance — resulting in operational inefficiencies and poor data reliability.

Key gaps identified in the current hostel management process:

- No centralised digital record system
- Human error prevalent in manual registers
- No structured fee tracking or payment history
- No visitor log management or audit trail
- Maintenance requests untracked and unresolved

3.3 Proposed Solution

CampusNest is a console-based C application that addresses all identified gaps through a modular, menu-driven system. Built entirely in pure C99 with no external libraries or frameworks, it uses file persistence through .txt files to ensure all data survives program restarts without any database requirement.

Feature	Details
C Language (C99)	Pure C99 standard. No external libraries or frameworks — appropriate for coursework level.
File Persistence	All data saved to plain .txt files. Data survives program restarts with no database required.
Modular Design	5 independent modules, each with its own .c source file and clearly defined responsibilities.
Input Validation	fgets + sscanf pattern used throughout to prevent buffer overflow and handle invalid input.

3.4 Project Objectives

No.	Module	Objective
01	Student Management	Register, track, search, and deactivate hostel residents.
02	Room Management	Configure rooms, manage capacity, and track allocations.
03	Fee Management	Record monthly fees, mark payments, view outstanding dues.
04	Visitor Management	Log all visitor entries with contact and purpose details.
05	Maintenance Tracking	Raise, monitor, and resolve hostel maintenance requests.

3.5 System Architecture

CampusNest follows a three-layer architecture that separates user interaction, application logic, and data persistence. The main.c file acts as the navigation controller, routing user input to the appropriate module, while each .c file handles its own business logic independently.

Layer	Components
Presentation Layer	User / Administrator → Main Controller (main.c) – Menu & Navigation via switch-case
Application Logic	student.c · room.c · fee.c · visitor.c · maintenance.c
Data Storage Layer	students.txt · rooms.txt · fees.txt · visitors.txt · requests.txt

4. Programming Concepts Used

4.1 Technical Stack

The project applies the following C programming concepts:

Concept	Implementation in CampusNest
C Language (C99)	Core language with structs, arrays, pointers, and standard I/O used throughout all modules.
Structures	5 typedef structs defined: Student, Room, FeeRecord, Visitor, MaintenanceRequest (one per member).
File Handling	fopen / fclose / fprintf / fscanf for append-and-read persistence across all 5 data files.
Functions	Minimum 25 logical functions total (5 per member); single-responsibility per function design.
Arrays of Structures	Each module loads and processes arrays of its respective struct type for batch operations.
Control Flow	while loop combined with switch-case for menu-driven, safe interaction in main.c.
Input Validation	fgets + sscanf pattern prevents buffer overflow; range checks reject invalid inputs.
Sorting & Searching	Bubble sort and linear search implemented per module for ID, name, amount, date, and priority.
String Handling	strcmp, strcpy, strcat used for name matching, email and contact field operations.
Pointers	Passed to functions for in-place struct field modification and file stream management.

4.2 Structure Definitions

Each member defined at least one structure. The five structures together represent all core data entities of the system:

```
/* STRUCTURE 1 – Student (Member 1) */
typedef struct {
    int id;
    char name[NAME_LEN];
    int age;
    char gender[GENDER_LEN];
    char contact[CONTACT_LEN];
    char email[EMAIL_LEN];
    int roomNo;
    char enrollDate[DATE_LEN];
} Student;

/* STRUCTURE 2 – Room (Member 2) */
typedef struct {
    int roomNo;
    int capacity;
    int occupied;
    char type[NAME_LEN]; /* Single / Double / Triple */
    char status[STATUS_LEN]; /* Available / Full / Maintenance */
    float pricePerMonth;
} Room;

/* STRUCTURE 3 – Fee (Member 3) */
typedef struct {
    int feeId;
    int studentId;
    float amount;
    char month[DATE_LEN];
    char status[STATUS_LEN]; /* Paid / Unpaid / Partial */
    char payDate[DATE_LEN];
} Fee;

/* STRUCTURE 4 – Visitor (Member 4) */
typedef struct {
    int visitorId;
    char visitorName[NAME_LEN];
    int studentId;
    char relation[NAME_LEN];
    char checkIn[TIME_LEN];
    char checkOut[TIME_LEN];
    char visitDate[DATE_LEN];
} Visitor;

/* STRUCTURE 5 – MaintenanceRequest (Member 5) */
typedef struct {
    int reqId;
    int roomNo;
    char issue[ISSUE_LEN];
    char priority[STATUS_LEN]; /* Low / Medium / High */
    char status[STATUS_LEN]; /* Pending / In-Progress / Resolved */
    char reportDate[DATE_LEN];
} MaintenanceRequest;
```

4.3 Functions by Module

Each member implemented a minimum of 5 logical functions, producing 30 functions across the full project:

Module 1 — Student Management (254156M – Perera M. I. P)

- addStudent() – Register a new student with full personal and room details
- viewAllStudents() – Display all active student records in tabular format
- searchStudent() – Search records by student ID or by full name
- updateRoomAssignment() – Update the room number assigned to a student
- deleteStudent() – Remove a student record by ID with confirmation
- sortStudentsById() – Sort the student array by ID using bubble sort

Module 2 — Room Management (254157R – Perera O. S. G)

- addRoom() – Add a new room with type, capacity, and monthly price
- viewAllRooms() – Display all rooms with full occupancy and status
- searchRoom() – Find a specific room by room number or room type
- updateRoomStatus() – Change a room's availability status
- viewAvailableRooms() – Filter and list only rooms currently available
- sortRooms() – Sort rooms by number; deleteRoom() – Remove a room record

Module 3 — Fee Management (254158V – Perera P.B.H.N.S)

- addFeeRecord() – Add a new monthly fee entry for a student
- viewAllFees() – Display the complete fee register
- searchFeeByStudentId() – Retrieve all fee records for a given student
- updateFeeStatus() – Mark a fee entry as Paid or Unpaid
- feeSummary() – Report total collected vs outstanding balance
- sortFeesByAmount() – Sort fee records in ascending order by amount

Module 4 — Visitor Management (254159B – Perera P.S.K)

- logVisitor() – Register a visitor entry with purpose and contact details
- viewAllVisitors() – Display the complete visitor log
- searchVisitorByStudentId() – Find all visitors associated with a student
- visitorCountReport() – Count and report total visitor statistics
- sortVisitorsByDate() – Sort visitor records in chronological order
- deleteVisitorRecord() – Remove a visitor entry from the log

Module 5 — Maintenance Management (254160U – Perera R.K.O.C)

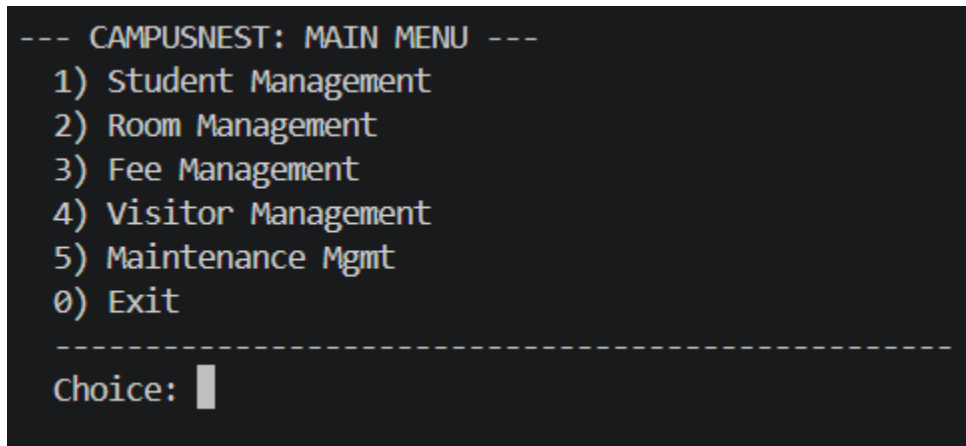
- addRequest() – Log a new maintenance request with issue and priority
- viewAllRequests() – Display all maintenance requests with full details
- updateRequestStatus() – Change status to In Progress or Resolved
- searchByRoom() – Find all maintenance requests for a specific room
- maintenanceSummary() – Report count of pending vs resolved requests
- sortByPriority() – Sort by High/Medium/Low; deleteRequest() – Remove entry

5. Screenshots of System Output

The following console output captures demonstrate that all five modules are fully functional and produce well-structured output. Each figure is labelled with the corresponding module and contributing member.

5.1 Main Menu Interface

The main menu is driven by a while loop and switch-case in main.c, routing user input to the appropriate module.



```
--- CAMPUSNEST: MAIN MENU ---
1) Student Management
2) Room Management
3) Fee Management
4) Visitor Management
5) Maintenance Mgmt
0) Exit
-----
Choice: █
```

Figure 1 – CampusNest Main Menu Interface

5.2 Student Management (254156M – Perera M. I. P)

Student registration captures full personal details including contact, email, assigned room, and enrolment date. The module supports Add, View, Search, Update, Delete, and Sort operations.

```
=== MEMBER 1: STUDENT MANAGEMENT ===
```

- 1) Add Student
- 2) View All Students
- 3) Search Student
- 4) Update Room Assignment
- 5) Delete Student
- 6) Sort by ID
- 0) Back to Main Menu

```
-----
```

```
Choice: 3
```

```
Search by: 1) ID    2) Name
```

```
Choice: 1
```

```
Enter ID: 254157
```

```
ID      : 254157
```

```
Name    : savindu perera
```

```
Age     : 20
```

```
Gender  : M
```

```
Contact : 0705921490
```

```
Email   : savinduperera@gmail.com
```

```
Room    : 12
```

```
Enrolled: 2024/06/08
```

Figure 2 – Student Registration Input

```
=== MEMBER 1: STUDENT MANAGEMENT ===
```

- 1) Add Student
 - 2) View All Students
 - 3) Search Student
 - 4) Update Room Assignment
 - 5) Delete Student
 - 6) Sort by ID
 - 0) Back to Main Menu
-

Choice: 3

Search by: 1) ID 2) Name

Choice: 1

Enter ID: 254157

ID : 254157

Name : savindu perera

Age : 20

Gender : M

Contact : 0705921490

Email : savinduperera@gmail.com

Room : 12

Enrolled: 2024/06/08

Figure 3 – Student Search by ID

```
=== MEMBER 1: STUDENT MANAGEMENT ===
```

- 1) Add Student
 - 2) View All Students
 - 3) Search Student
 - 4) Update Room Assignment
 - 5) Delete Student
 - 6) Sort by ID
 - 0) Back to Main Menu
-

Choice: 5

Enter Student ID to delete: 254157

[+] Student deleted.

5.3 Room Management (254157R – Perera O. S. G)

The room module tracks room number, type, capacity, occupancy, status, and monthly price with tabular display.

```
=== MEMBER 2: ROOM MANAGEMENT ===
1) Add Room
2) View All Rooms
3) Search Room
4) Update Room Status
5) View Available Rooms
6) Sort Rooms
7) Delete Room
0) Back to Main Menu
-----
Choice: 1
Room Number : 65
Type (Single/Double/Triple): Single
Capacity : 2
Price/Month : 10000
[+] Room 65 added.

=== MEMBER 2: ROOM MANAGEMENT ===
1) Add Room
2) View All Rooms
3) Search Room
4) Update Room Status
5) View Available Rooms
6) Sort Rooms
7) Delete Room
0) Back to Main Menu
-----
Choice: 1
Room Number : 70
Type (Single/Double/Triple): Single
Capacity : 2
Price/Month : 10000
[+] Room 70 added.
```

Room	Type	Capacity	Occupied	Status	Price/Month
65	Single	2	0	Available	10000.00
70	Single	2	0	Available	10000.00

Total rooms: 2					

Figure 5 – Room Registration and View All Rooms

5.4 Fee Management (254158V – Perera P.B.H.N.S)

The fee module records monthly hostel fees per student, supports payment marking, and generates summary reports.

```
=== MEMBER 3: FEE MANAGEMENT ===
1) Add Fee Record
2) View All Fees
3) Search by Student ID
4) Update Fee Status
5) Fee Summary
6) Sort by Amount
0) Back to Main Menu
-----
Choice: 1
Student ID : 254158
Amount (LKR) : 10000
Month (e.g. Jan-2025): April-2026
Status (Paid/Unpaid/Partial): Paid
Payment Date : 2024/04/09
[+] Fee record added. Fee ID: 1000

=== MEMBER 3: FEE MANAGEMENT ===
1) Add Fee Record
2) View All Fees
3) Search by Student ID
4) Update Fee Status
5) Fee Summary
6) Sort by Amount
0) Back to Main Menu
-----
Choice: 5

--- Fee Summary ---
Total records : 1
Total billed : LKR 10000.00
Total collected : LKR 10000.00
Outstanding : LKR 0.00
Unpaid records : 0
```

5.5 Visitor Management (254159B – Perera P.S.K)

The visitor module maintains a complete log of all hostel visitors including name, contact, purpose, associated student, and visit date.

```
--- CAMPUSNEST: MAIN MENU ---
1) Student Management
2) Room Management
3) Fee Management
4) Visitor Management
5) Maintenance Mgmt
0) Exit
-----
Choice: 4

=== MEMBER 4: VISITOR MANAGEMENT ===
1) Log Visitor
2) View All Visitors
3) Search by Student ID
4) Visitor Count Report
5) Sort by Date
6) Delete Visitor Record
0) Back to Main Menu
-----
Choice: 1
Visitor Name   : Kamal perera
Student ID    : 254158
Relation      : Mother
Visit Date    : 2026/05/03
Check-In Time : 08:30
Check-Out Time: 17:30
[+] Visitor logged. Visitor ID: 2000
```

Figure 7 – Visitor Log Entry and Count Report

5.6 Maintenance Management (254160U – Perera R.K.O.C)

The maintenance module handles the full request lifecycle: logging with priority, tracking through In Progress to Resolved, and summary reporting.

```
=== MEMBER 5: MAINTENANCE MANAGEMENT ===
1) Add Request
2) View All Requests
3) Update Request Status
4) Search by Room
5) Maintenance Summary
6) Sort by Priority
7) Delete Request
0) Back to Main Menu
-----
Choice: 1
Room Number   : 65
Issue          : The door to the room is broken
Priority (Low/Medium/High): High
Report Date    : 2026/06/02
[+] Request logged. Request ID: 3000
```

Figure 8 – Maintenance Request Entry and Summary

6. Challenges Faced and Solutions

The following table summarises the key technical challenges encountered during development and the solutions implemented by the group:

Challenge	Solution Implemented
Input buffer conflicts with scanf / fgets	Replaced all string scanf calls with fgets + sscanf. Buffers are flushed between inputs to prevent ghost reads and overflow.
File data corruption on repeated writes	Switched to append mode ('a') for writing and sequential fscanf for reading. Load functions rebuild the array from file on each run.
Coordinating five modules in one project	Each module uses a separate .c file with consistent naming conventions. main.c routes between modules using a top-level switch-case.
Deletion without database support	Rewrite-on-delete: load all records into array, remove the target entry, then rewrite the entire file to maintain data integrity.
Sorting structs without standard library sort	Bubble sort implemented on arrays of structs using a temporary struct variable for swapping and custom per-field comparators.
Verifying deletion correctness	Post-deletion search confirms the record no longer exists; the system returns "No matching student found" as verification.
Input validation for numeric fields	sscanf return value checking combined with range validation loops reject non-numeric and out-of-range inputs before processing.

7. System Benefits

CampusNest delivers the following operational improvements over the previous manual paper-based system:

Centralised Records

All hostel data in one system — students, rooms, fees, visitors, and maintenance — replacing fragmented paper registers.

Faster Operations

Instant search, allocation, and report generation through the menu-driven interface, replacing hours of manual lookup.

Reduced Human Error

Structured input and type validation eliminates the data inconsistency common in manual entry.

Data Persistence

Text-file storage ensures all records survive program restarts without requiring a database server.

Audit Trail

All fee payments, visitor entries, and maintenance requests are permanently recorded and retrievable.

Easy Maintainability

Modular .c files allow updates to one module without affecting any other part of the system.

8. Conclusion

CampusNest is a console-based hostel management system developed to simplify the management of students, rooms, fees, visitors, and maintenance requests. The project successfully applies key C programming concepts — structures, functions, arrays, searching, sorting, and input validation — to solve a real-world problem.

All five group members contributed equally, each taking ownership of one complete module from design through implementation and testing. The system meets all mandatory requirements: 5 structures (one per member), at least 25 logical functions (minimum 5 per member), a menu-driven interface using switch-case, arrays of structures, string handling, proper input/output handling, file persistence, and meaningful real-world data.

The modular design of CampusNest ensures each component can be maintained and extended independently, providing a scalable foundation that could, in future iterations, support database connectivity or graphical interfaces beyond the current implementation scope.
